

HMC-LLM-LITE: Reranking Semântico com LLM para Classificação Hierárquica de Documentos Científicos

Proposta arquitetural baseada no modo `globalLLMLite` do `hmc-torch`

Bruno Sette

Universidade Federal de Minas Gerais — UFMG

`bruno.sette@dcc.ufmg.br`

Junho de 2026

Abstract

A classificação hierárquica de texto exige que um documento seja associado a rótulos organizados em uma taxonomia, preservando relações de ancestralidade entre classes gerais e específicas. Em coleções científicas, como ArXiv e Web of Science, essa tarefa é particularmente sensível a ambiguidades semânticas: artigos próximos podem pertencer a subáreas distintas, e pequenos erros nos rótulos folha afetam a coerência da predição final. Este artigo propõe a HMC-LLM-LITE, uma arquitetura híbrida que combina um classificador supervisionado hierárquico com um estágio leve de reranking por modelo de linguagem. O classificador base, treinado end-to-end com um encoder científico SPECTER2, gera scores para toda a hierarquia e aplica restrições de ancestralidade via matriz R . Em seguida, apenas amostras incertas são enviadas a um LLM externo, que recebe o texto do documento, um conjunto pequeno de candidatos e seus caminhos hierárquicos, retornando uma seleção estruturada em JSON. A proposta mantém o classificador neural como mecanismo principal de recuperação de rótulos e usa o LLM somente como verificador semântico local, reduzindo custo, latência e risco de alucinação de classes fora da taxonomia.

Palavras-chave: classificação hierárquica, HTC, HMC, SPECTER2, LLM, reranking, Groq, ArXiv, WOS.

1 Introdução

A classificação hierárquica de texto (HTC) generaliza a classificação multi-rótulo ao impor uma estrutura taxonômica sobre o espaço de saída. Em vez de prever classes independentes, o modelo precisa produzir um conjunto de rótulos consistente: se um documento é classificado como `cs.LG`, ele também deve ser compatível com a área pai `cs`. Essa restrição é natural em coleções científicas, nas quais áreas e subáreas organizam o conhecimento de forma explícita.

Modelos neurais modernos para HTC normalmente seguem uma de três estratégias: restrição direta sobre as saídas, propagação de informação no grafo de rótulos, ou adaptação do encoder textual à hierarquia. O repositório `hmc-torch` implementa essas alternativas por meio dos métodos `global`, `globalE2E` e `globalSOTA`. A proposta deste artigo adiciona uma quarta peça: usar um LLM como reranker semântico apenas nos casos em que o classificador supervisionado está incerto.

A motivação é pragmática. LLMs são fortes em interpretação semântica local e em comparação textual entre alternativas, mas são caros e pouco adequados para explorar diretamente centenas de rótulos. Por outro lado, classificadores supervisionados são eficientes para varrer todo o espaço de saída, mas podem confundir subáreas semanticamente próximas. A HMC-LLM-LITE combina

esses pontos fortes: o classificador recupera candidatos prováveis e o LLM decide apenas entre esses candidatos.

2 Contribuições

Este artigo organiza a proposta em quatro contribuições principais:

1. **Arquitetura híbrida para HTC:** uma combinação entre encoder científico, classificador hierárquico supervisionado e reranking semântico por LLM.
2. **Gate de incerteza:** um critério simples baseado no score máximo, na margem entre os melhores candidatos e no parâmetro top- k , evitando chamadas desnecessárias ao LLM.
3. **Prompt restrito por taxonomia:** o LLM recebe apenas rótulos candidatos válidos, pontuações e caminhos hierárquicos, reduzindo a chance de saída fora do espaço de classes.
4. **Implementação reproduzível:** respostas em JSON e cache local tornam o reranking auditável e evitam recomputação de chamadas já feitas.

3 Formulação do problema

Seja d um documento científico composto por título e resumo. Seja $\mathcal{Y} = \{y_1, \dots, y_N\}$ o conjunto de rótulos da taxonomia, excluindo a raiz. A tarefa consiste em estimar um vetor multi-rótulo $\hat{\mathbf{y}} \in [0, 1]^N$, respeitando as relações de ancestralidade entre classes.

O classificador neural produz logits $\mathbf{z} = f_\theta(d)$ e probabilidades $\mathbf{p} = \sigma(\mathbf{z})$. A consistência hierárquica é aplicada por uma matriz de ancestralidade R , que codifica quais nós são ancestrais de outros nós. De forma abstrata, a saída restrita é:

$$\mathbf{s} = g_R(\mathbf{p}), \quad (1)$$

onde g_R representa a função de pós-processamento hierárquico que garante compatibilidade entre filho e ancestral. A HMC-LLM-LITE atua depois desse passo, transformando $\mathbf{s}$ em um conjunto reduzido de candidatos.

Para cada documento, define-se:

$$C_k(d) = \text{TopK}(\mathbf{s}, k), \quad (2)$$

onde $C_k(d)$ é o conjunto dos k rótulos avaliáveis com maior score. O reranking por LLM é ativado apenas se a amostra for incerta:

$$s_{(1)} < \tau \quad \text{ou} \quad s_{(1)} - s_{(2)} < \delta, \quad (3)$$

em que $s_{(1)}$ e $s_{(2)}$ são os dois maiores scores, τ é o limiar de confiança e δ é a margem mínima entre os dois melhores candidatos. Quando o classificador está confiante, a predição original é preservada.

4 Arquitetura proposta

A Figura 1 mostra o desenho da proposta. O pipeline começa com um encoder científico, adequado para documentos acadêmicos. No modo atual do `hmc-torch`, o encoder padrão é `allenai/specter2_base`; o texto de entrada é construído a partir do título e do resumo no ArXiv, ou do resumo no WOS.

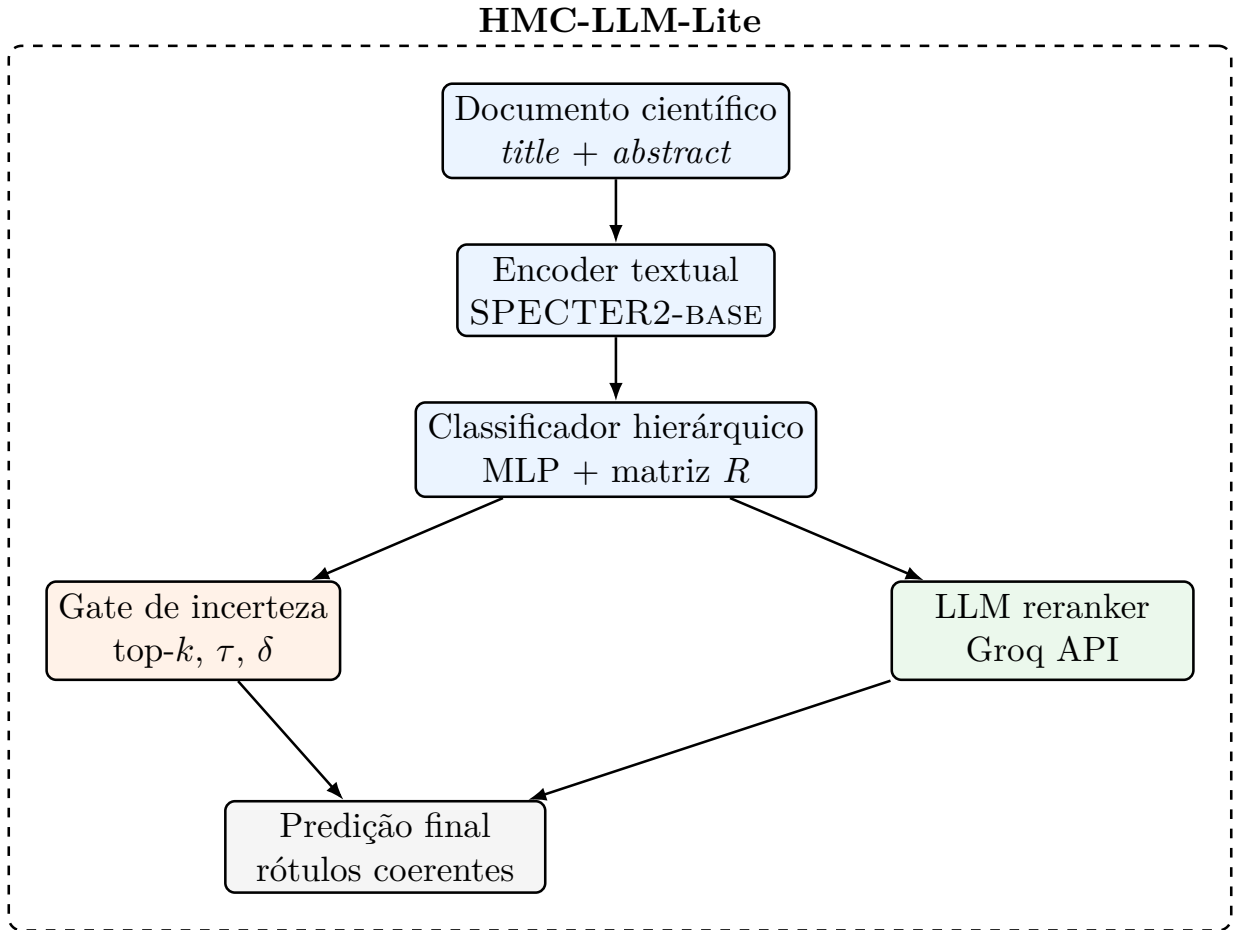


Figure 1: Visão geral da HMC-LLM-LITE. O classificador supervisionado continua sendo responsável por recuperar candidatos em toda a hierarquia. O LLM atua somente como reranker sobre uma lista curta de rótulos candidatos.

O segundo bloco é o classificador hierárquico. Na variante implementada, a base é o modelo **globalE2E**: o transformer é ajustado junto com uma cabeça MLP, e a saída passa pela matriz R para preservar coerência entre ancestrais e descendentes. Isso evita que o LLM precise inferir a hierarquia do zero.

O terceiro bloco é o gate de incerteza. Ele decide se a amostra deve ser enviada ao LLM. Se o maior score já é alto e a margem para o segundo candidato é suficiente, a predição supervisionada é aceita sem custo adicional. Caso contrário, os top- k candidatos são formatados em um prompt estruturado.

5 Reranking por LLM

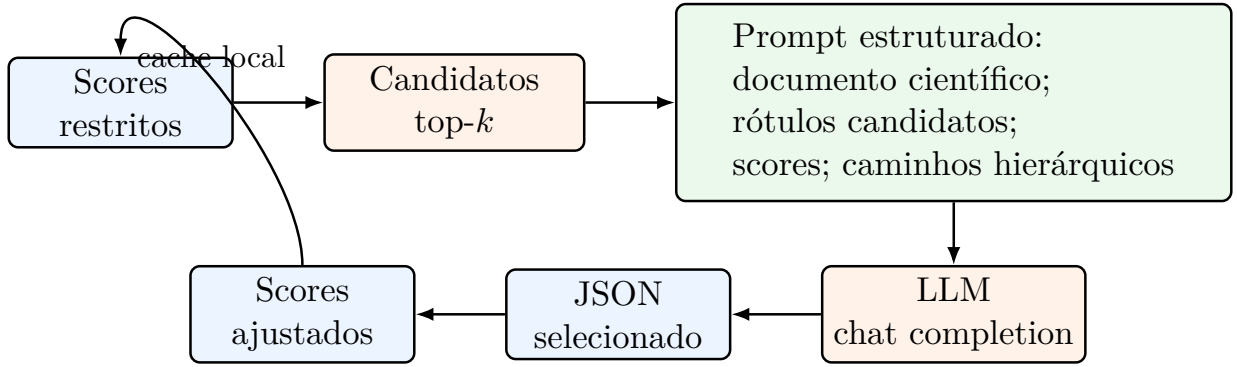


Figure 2: Detalhe do reranking. O LLM recebe apenas candidatos produzidos pelo classificador, retorna JSON e nunca precisa gerar livremente nomes de classes.

O reranking segue um contrato simples. Para cada documento incerto, o sistema constrói uma lista de candidatos:

$$c_j = (\text{label}_j, \text{score}_j, \text{path}_j). \quad (4)$$

O campo *path* contém o caminho hierárquico do rótulo, por exemplo `cs > cs.LG`. O LLM recebe essa lista e deve retornar um objeto JSON com o campo `selected_labels`. A predição final preserva os scores dos rótulos selecionados e zera os candidatos rejeitados:

$$s'_i = \begin{cases} s_i, & \text{se } y_i \in S_{\text{LLM}}, \\ 0, & \text{se } y_i \in C_k(d) \setminus S_{\text{LLM}}, \\ s_i, & \text{se } y_i \notin C_k(d). \end{cases} \quad (5)$$

Essa regra é conservadora: o LLM não adiciona rótulos fora da lista proposta pelo classificador. Ele apenas remove candidatos semanticamente fracos dentro da lista curta. Assim, a arquitetura reduz o espaço de decisão do LLM e preserva o aprendizado supervisionado como fonte principal de recall.

6 Algoritmo

O procedimento executado pela HMC-LLM-LITE pode ser resumido da seguinte forma:

1. Treinar o modelo base **globalE2E** com encoder científico, cabeça MLP e restrição hierárquica.
2. Para cada documento de teste, calcular os scores restritos $\mathbf{s} = g_R(\sigma(f_\theta(d)))$.

3. Selecionar os top- k rótulos avaliáveis.
4. Se $s_{(1)} \geq \tau$ e $s_{(1)} - s_{(2)} \geq \delta$, aceitar a predição do classificador.
5. Caso contrário, enviar documento e candidatos ao LLM.
6. Interpretar a resposta JSON e manter apenas os candidatos selecionados.
7. Aplicar as métricas hierárquicas sobre os scores ajustados.

Na implementação do `hmc-torch`, esse procedimento é exposto pelo argumento:

```
python -m hmc.main \
  --dataset_name arxiv \
  --method globalLLMLite \
  --device cuda \
  --dataset_path ./data \
  --output_path ./output
```

7 Diferença para uso direto de LLM

Uma alternativa simples seria pedir ao LLM que classifique diretamente cada documento dentro da taxonomia. Essa opção é menos controlada por três motivos. Primeiro, o LLM precisaria considerar dezenas ou centenas de rótulos em cada chamada. Segundo, a resposta poderia conter classes inexistentes, nomes parafraseados ou níveis incompletos da hierarquia. Terceiro, o custo cresceria linearmente com o número de documentos, mesmo nos casos em que o classificador supervisionado já está confiante.

A HMC-LLM-LITE evita esses problemas ao decompor a decisão. O classificador neural faz a recuperação ampla e barata. O LLM entra apenas como árbitro local em regiões ambíguas do espaço de rótulos. Essa separação é especialmente adequada para taxonomias científicas, onde muitas confusões relevantes ocorrem entre subáreas próximas, como aprendizado de máquina, inteligência artificial, estatística computacional e otimização.

8 Implementação

A implementação atual é composta por três módulos principais:

Componente	Arquivo	Função
Entrada principal	<code>src/hmc/main.py</code>	Seleciona <code>globalLLMLite</code>
Pipeline LLM	<code>llm_train.py</code>	Treina E2E e aplica postprocess
Cliente Groq	<code>groq_agent.py</code>	Prompt, cache e parse JSON

Table 1: Componentes da implementação da HMC-LLM-LITE no `hmc-torch`.

O modo `globalLLMLite` chama internamente o treinamento `globalLLM`, mas fixa uma política mais econômica: o reranking é aplicado apenas a amostras incertas e a resposta do LLM é curta. Os principais parâmetros são `llm_top_k`, `llm_margin`, `llm_confidence_threshold`, `llm_max_tokens`, `llm_temperature`, `llm_cache_dir`, `llm_k_max`, `llm_expand_hierarchy`, `llm_max_calls`, `llm_timeout` e `llm_fallback_on_error`. O modelo padrão de LLM é configurável; no código atual, o valor inicial é `llama-3.1-8b-instant`.

O cache local é importante para reprodutibilidade. A chave de cache inclui o texto do documento, os candidatos, o modelo LLM, a temperatura, o número máximo de tokens e o caminho

hierárquico. Assim, uma mesma chamada não precisa ser reexecutada em rodadas posteriores. As estatísticas do reranking são salvas junto com o arquivo `test-scores.json`, incluindo número de chamadas, falhas, *fallbacks*, amostras ignoradas por confiança e *recall@k* dos candidatos.

9 Protocolo experimental sugerido

A avaliação deve comparar a HMC-LLM-LITE contra três baselines do próprio repositório:

1. **global**: encoder congelado, embeddings pré-computados e MLP com restrição hierárquica.
2. **globalE2E**: ajuste end-to-end do transformer com cabeça global.
3. **globalSOTA**: transformer ajustado com GNN sobre o grafo de rótulos.

As métricas principais devem incluir Micro-F1, Macro-F1, precisão e recall por nível hierárquico. Também é necessário reportar métricas operacionais: percentual de documentos enviados ao LLM, número total de chamadas, custo estimado, latência média e taxa de respostas inválidas ou vazias. Para evitar otimização direta no conjunto de teste, o protocolo do artigo usa limiar fixo de decisão em 0,5 por padrão; a busca por melhor limiar deve ser mantida desativada, exceto em ablações explicitamente identificadas.

Método	Encoder	Hierarquia	LLM
global	Congelado	Matriz R	Não
globalE2E	Ajustado	Matriz R	Não
globalSOTA	Ajustado	Matriz R + GNN	Não
globalLLMLite	Ajustado	Matriz R	Gate incerto

Table 2: Comparação entre regimes experimentais no `hmc-torch`.

10 Hipóteses

A proposta deve ser avaliada a partir de três hipóteses:

1. **H1**: o reranking melhora a precisão em rótulos folha, pois remove candidatos semanticamente plausíveis mas incorretos.
2. **H2**: o ganho é maior em documentos incertos, isto é, quando a margem entre os principais candidatos é pequena.
3. **H3**: o custo permanece controlado porque apenas uma fração dos documentos aciona o LLM.

Essas hipóteses também indicam os riscos. Um LLM muito conservador pode aumentar precisão e reduzir recall. Um gate muito agressivo pode enviar documentos demais ao LLM e tornar o método caro. Por isso, τ , δ e k devem ser tratados como hiperparâmetros, não como constantes universais.

11 Limitações e mitigações

As limitações principais da HMC-LLM-LITE podem ser tratadas dentro da própria arquitetura. A primeira é a dependência do top- k : se o rótulo correto não entra na lista de candidatos, o LLM não consegue selecioná-lo. Para reduzir esse risco, o gate passa a usar uma seleção adaptativa

em vez de um k fixo. Em documentos de baixa confiança, o sistema expande a lista até $k_{\max}$ e inclui também rótulos estruturalmente próximos: pais, filhos e irmãos dos candidatos mais bem pontuados. Assim, o LLM continua operando sobre um espaço pequeno, mas com maior chance de conter a classe correta.

$$C(d) = \text{TopK}(\mathbf{s}, k(d)) \cup \text{Anc}(C) \cup \text{Child}(C) \cup \text{Sib}(C), \quad (6)$$

onde $k(d)$ cresce com a incerteza do classificador e é limitado por $k_{\max}$. Na avaliação, esse mecanismo deve ser acompanhado pela métrica $\text{recall}@k$ dos candidatos, pois ela mede se o gargalo está no recuperador neural ou no reranker.

A segunda limitação envolve custo, latência e dependência de um provedor externo. Ela é mitigada por quatro mecanismos: chamadas apenas para amostras incertas, cache determinístico por conteúdo do prompt, *fallback* automático para a predição supervisionada em caso de erro ou timeout, e orçamento máximo de chamadas por execução. Com isso, o LLM deixa de ser uma dependência crítica do pipeline: quando a chamada falha, o sistema mantém a saída do classificador base.

Risco	Mitigação	Efeito esperado
Rótulo fora do top- k	Top- k adaptativo + vizinhança hierárquica	Maior recall de candidatos
Custo elevado	Gate de incerteza + cache	Menos chamadas externas
Latência	Cache + limite de chamadas	Tempo controlado por execução
Resposta inválida	Validação JSON + fallback	Robustez operacional
Variação do provedor	Modelo versionado + cache congelado	Reprodutibilidade

Table 3: Mitigações incorporadas à proposta para tornar o uso de LLM controlado e reprodutível.

A terceira limitação é que o reranking, isoladamente, atua apenas no pós-processamento. A solução proposta é transformar as decisões do LLM em sinal fraco de treinamento. As respostas aceitas podem alimentar um conjunto auxiliar de pares positivos e negativos: candidatos selecionados pelo LLM são tratados como positivos, enquanto candidatos rejeitados dentro do mesmo top- k são tratados como negativos difíceis. Esse conjunto permite treinar um reranker local menor ou calibrar os limiares τ e δ sem depender do LLM em todas as execuções futuras.

Para o primeiro experimento, a arquitetura opera no modo **online**: o LLM é usado diretamente no pós-processamento das amostras incertas. Como extensão posterior, o histórico de decisões do LLM pode alimentar um modo **destilado**, no qual um módulo local de calibração aproxima o comportamento do reranker externo e reduz chamadas em rodadas futuras. Essa estratégia preserva o ganho semântico do LLM, mas move parte do custo para uma etapa offline e reaproveitável.

12 Conclusão

Este artigo apresentou a HMC-LLM-LITE, uma proposta de arquitetura para classificação hierárquica de documentos científicos que combina aprendizado supervisionado com reranking semântico por LLM. O desenho preserva o classificador neural como mecanismo principal de recuperação e usa o LLM apenas em casos incertos, sobre um conjunto restrito de candidatos válidos. Essa escolha busca capturar o melhor dos dois regimes: eficiência e cobertura do classificador, interpretação semântica local do LLM e coerência estrutural da matriz hierárquica.

No contexto do **hmc-torch**, a proposta é implementada pelo método **globalLLMLite** e pode ser avaliada diretamente nos datasets ArXiv e WOS. Com as mitigações propostas, o método deixa de depender de um top- k rígido, controla custo por gate e cache, mantém fallback determinístico para falhas externas e cria um caminho de destilação para reaproveitar decisões do LLM no

treinamento futuro. O primeiro experimento deve medir simultaneamente ganho de F1, recall@k dos candidatos, redução de erros em rótulos folha e custo médio por documento.

Referências

1. Cohan, A. et al. *SPECTER: Document-level Representation Learning using Citation-informed Transformers*. ACL, 2020.
2. Giunchiglia, E. and Lukasiewicz, T. *Coherent Hierarchical Multi-label Classification Networks*. NeurIPS, 2020.
3. Zhou, J. et al. *Hierarchy-Aware Global Model for Hierarchical Text Classification*. ACL, 2020.
4. Wang, Z. et al. *Incorporating Hierarchy into Text Encoder: a Contrastive Learning Approach for Hierarchical Text Classification*. ACL, 2022.
5. Wang, Z. et al. *HPT: Hierarchy-aware Prompt Tuning for Hierarchical Text Classification*. EMNLP, 2022.